

Proyecto 1 - Entrega parcial

Proyecto 1 — Entrega parcial

8. Especificación del servidor MCP desarrollado

8.1 Propósito y caso de uso

8.2 Transporte y protocolo

8.3 Endpoints

8.4 Herramientas y parámetros

8.5 Formato de respuesta

8.6 Manejo de errores

8.7 Ejemplos de intercambio

8.8 Ejecución y conexión

8.9 Persistencia

10. Conclusiones y comentarios sobre el proyecto

10.1 Estado actual

10.2 Decisiones de diseño que resultaron acertadas

10.3 Dificultades encontradas

10.4 Lo que falta

10.5 Validación con hosts de terceros

10.6 El bucle agéntico de extremo a extremo

10.7 Comentario sobre MCP

Proyecto 1 — Entrega parcial

Curso: CC3067 Redes **Universidad del Valle de Guatemala Repositorio:**

<https://github.com/andresm220/Proyecto1Redes> **Servidor MCP desarrollado:** netops

v1.0.0 Versión de protocolo implementada: 2025-11-25

8. Especificación del servidor MCP desarrollado

La especificación completa, con los esquemas íntegros y los intercambios JSON-RPC crudos, está versionada en el repositorio en `servers/netops/SPEC.md`. Esta sección la resume y explica las decisiones detrás de ella.

8.1 Propósito y caso de uso

netops modela el back office de la mesa de soporte técnico de un proveedor de internet. Expone las operaciones que un agente de primera línea realiza mientras tiene al abonado en el teléfono: identificar la cuenta, leer el estado actual del enlace, verificar si una

incidencia masiva ya explica el reclamo, correr un diagnóstico, y abrir o dar seguimiento a un ticket.

El caso de uso se eligió porque su valor está en el **encadenamiento**, no en una consulta aislada. Un reclamo de “no tengo internet” no se resuelve con una sola llamada: hay que identificar al abonado, leer su enlace y revisar las incidencias de su región antes de decidir si un ticket siquiera corresponde. Eso lo vuelve un caso apropiado para un host gobernado por un LLM, que puede encadenar las llamadas y razonar sobre los resultados.

Los datos son simulados pero **determinísticos**: las métricas del enlace (latencia, pérdida de paquetes, SNR) se derivan de un hash del identificador de cuenta en lugar de un generador aleatorio, de modo que la misma llamada devuelve siempre la misma lectura. Los tickets y visitas, en cambio, son estado persistente real que sobrevive al reinicio del proceso.

Los datos semilla están contruidos de forma que las entidades se relacionen entre sí. La cuenta GT-10233 está en la región peten, donde la incidencia OUT-2026-013 está activa; por eso check_service_status sobre esa cuenta reporta la incidencia en lugar de una falla individual. Sin esa relación, el modelo no tendría motivo para encadenar herramientas.

8.2 Transporte y protocolo

Aspecto	Valor
Protocolo	JSON-RPC 2.0
Versión MCP	2025-11-25
Transporte	stdio (el host lanza el servidor como subproceso)
Framing	NDJSON: un mensaje JSON por línea, terminado en \n
Codificación	UTF-8
stdout	Exclusivamente mensajes de protocolo
stderr	Exclusivamente logs del servidor, nunca parseado como protocolo

No se utilizó ningún SDK de MCP. Los sobres JSON-RPC, el handshake, la correlación de peticiones y el bucle de despacho están implementados a mano en host/mcp/jsonrpc.py y servers/netops/stdio_server.py. La única dependencia de terceros en el servidor es ninguna: corre con la biblioteca estándar de Python, lo cual se verificó ejecutándolo con un intérprete sin el entorno virtual.

El framing es NDJSON y **no** el encabezado Content-Length que utiliza LSP. Un mensaje no puede contener saltos de línea embebidos; json.dumps escapa los saltos dentro de las cadenas, por lo que una carga útil con saltos de línea permanece en una sola línea.

8.3 Endpoints

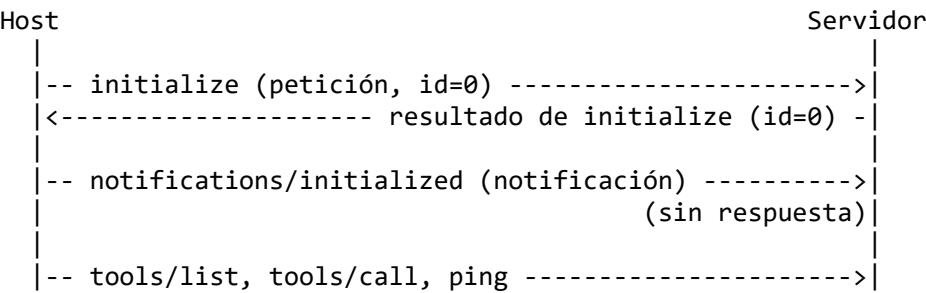
Sobre stdio no existen endpoints HTTP: no hay URLs ni rutas. El equivalente funcional son los **métodos JSON-RPC** que el servidor atiende, y el direccionamiento lo da el campo method del sobre en lugar de una ruta. Cuando el proyecto avance a la fase remota,

esos mismos métodos viajarán sobre un único endpoint HTTP, y la lógica de negocio no cambiará porque vive en un módulo independiente del transporte.

Método	Tipo	Requiere handshake	Descripción
initialize	petición	—	Negocia la versión de protocolo e identifica al cliente
notifications/initialized	notificación	—	El cliente confirma el handshake. No se responde nunca
ping	petición	No	Verificación de vida
tools/list	petición	Sí	Devuelve el catálogo de herramientas con sus esquemas
tools/call	petición	Sí	Ejecuta una herramienta

Cualquier otro método devuelve -32601.

Ciclo de vida



El handshake es obligatorio: tools/list y tools/call se rechazan con -32600 hasta que se complete. Si el servidor respondiera una versión de protocolo que el cliente no soporta, el cliente debe desconectarse y reportarlo en lugar de continuar; esa rama está implementada y cubierta por pruebas.

Ejemplo de handshake (capturado de una ejecución real):

```
{"jsonrpc": "2.0", "id": 0, "method": "initialize", "params": {"protocolVersion": "2025-11-25", "capabilities": {}, "clientInfo": {"name": "uvg-mcp-host", "version": "0.1.0"}}}

{"jsonrpc": "2.0", "id": 0, "result": {"protocolVersion": "2025-11-25", "capabilities": {"tools": {"listChanged": false}}, "serverInfo": {"name": "netops", "version": "1.0.0"}, "instructions": "Technical support back office for a Guatemalan ISP..."}}
```

El campo `instructions` es orientación redactada para el modelo. El host la incorpora a su `system prompt`, que es exactamente el propósito para el que el protocolo la define.

8.4 Herramientas y parámetros

Siete herramientas. Los esquemas completos están en `servers/netops/SPEC.md` sección 4; aquí se listan los parámetros y su validación.

`lookup_account`

Busca un abonado por identificador de cuenta o por teléfono.

Parámetro	Tipo	Obligatorio	Restricciones
<code>account_id</code>	string	Uno de los dos	<code>minLength: 3</code>
<code>phone</code>	string	Uno de los dos	<code>minLength: 8</code>

Debe suministrarse **exactamente uno** de los dos. Los teléfonos se comparan solo por sus dígitos, de modo que `+502 5555-0101`, `50255550101` y `(502) 5555 0101` resuelven a la misma cuenta.

Devuelve: plan, velocidad contratada, estado de la cuenta, tecnología, región, dirección de servicio y fecha de instalación.

`check_service_status`

Lee el estado actual del enlace de una cuenta.

Parámetro	Tipo	Obligatorio	Restricciones
<code>account_id</code>	string	Sí	<code>minLength: 3</code>

Devuelve: `link_state` (`up`, `degraded`, `down`, `not_provisioned`), `reason`, detalle legible, y un objeto `metrics` con latencia, pérdida de paquetes, SNR y velocidades medidas. Si una incidencia masiva cubre la región de la cuenta, incluye además `outage_id` y `eta`.

El campo `reason` se evalúa por prioridad: una suspensión administrativa pesa más que una incidencia masiva, y esta pesa más que una falla medida. Las métricas se devuelven siempre, incluso con el enlace caído, para que quien consulte pueda distinguir “sin señal” de “cuenta suspendida por saldo”.

`list_outages`

Lista incidencias masivas con su causa y tiempo estimado de reparación.

Parámetro	Tipo	Obligatorio	Restricciones
<code>region</code>	string	No	enum: guatemala, quetzaltenango, peten, escuintla
<code>active_only</code>	boolean	No	Por defecto <code>true</code>

Devuelve: conteo y arreglo de incidencias con outage_id, región, estado, causa, inicio, ETA, cuentas afectadas y servicios afectados.

Es la única herramienta sin modo de fallo de dominio: una región sin incidencias devuelve count: 0, que no es un error.

run_diagnostic

Ejecuta un diagnóstico contra la línea del abonado.

Parámetro	Tipo	Obligatorio	Restricciones
account_id	string	Sí	minLength: 3
test_type	string	Sí	enum: ping, speed, line

Devuelve: un objeto readings cuyo contenido depende del tipo de prueba, más un campo probable_cause siempre presente.

test_type	Campos de readings
ping	latency_ms, jitter_ms, packet_loss_pct, packets_sent
speed	downstream_mbps, upstream_mbps, contracted_mbps, pct_of_plan
line	snr_db, attenuation_db, technology, sync_errors_last_hour

open_ticket

Abre un ticket de soporte contra una cuenta.

Parámetro	Tipo	Obligatorio	Restricciones
account_id	string	Sí	minLength: 3
category	string	Sí	enum: connectivity, speed, billing, equipment, installation
description	string	Sí	minLength: 5, maxLength: 2000
priority	string	No	enum: low, normal, high, critical. Por defecto normal

Los identificadores son secuenciales y con relleno de ceros: TCK-00001. El ticket se escribe a disco **antes** de enviar la respuesta, por lo que un get_ticket posterior —en esta sesión o en otra— lo encuentra.

get_ticket

Lee el estado, el historial y la visita agendada de un ticket.

Parámetro	Tipo	Obligatorio	Restricciones
ticket_id	string	Sí	minLength: 5

Devuelve: el ticket completo, incluido un history de solo-anexado con entradas {at, status, note}.

schedule_visit

Agenda una visita técnica contra un ticket existente.

Parámetro	Tipo	Obligatorio	Restricciones
ticket_id	string	Sí	minLength: 5
date	string	Sí	pattern: ^\d{4}-\d{2}-\d{2}\$
time_window	string	Sí	enum: 08:00-12:00, 12:00-16:00, 16:00-20:00

Agendar sobre un ticket que ya tiene visita la reagenda, y la respuesta lo indica con "rescheduled": true.

8.5 Formato de respuesta

Toda llamada exitosa devuelve el mismo sobre:

```
{"content": [{"type": "text", "text": "<carga útil JSON>"}], "isError": false}
```

El bloque text contiene un objeto JSON indentado. Un fallo de dominio usa la misma forma con "isError": true y una carga útil {"error": "<mensaje>", ...contexto}.

8.6 Manejo de errores

La distinción entre error de protocolo y error de herramienta gobierna todo el diseño, y es probablemente la decisión más importante del servidor.

Un **error de protocolo** significa que el intercambio se rompió: el mensaje no pudo parsearse, el sobre era inválido, el método no existe, o los argumentos no corresponden al esquema publicado. Se reporta en el campo error de JSON-RPC y no hay result.

Un **error de herramienta** significa que el intercambio estuvo bien y la operación no pudo completarse: una cuenta que no existe, un ticket que nunca se abrió, una fecha en el pasado. Se reporta como un result **exitoso** cuyo campo isError vale true. Se espera que el modelo lo lea y reaccione, de modo que es un dato, no una falla de transporte.

Colapsar ambos rompería el host en las dos direcciones: una cuenta inexistente parecería una conexión caída, y una petición malformada parecería un resultado de negocio sobre el cual el modelo debería razonar.

Código	Nombre	Se emite cuando
-32700	Parse error	Una línea de stdin no es JSON válido. El id no es recuperable, por lo que la respuesta lleva "id": null — el único caso donde un id nulo es legal
-32600	Invalid Request	JSON válido pero sobre inválido; también si se llama tools/list o tools/call antes del handshake
-32601	Method not found	Cualquier método fuera de los cinco soportados
-32602	Invalid params	Falta params.name, la herramienta no existe, o algún argumento falla la validación del esquema
-32603	Internal error	Excepción no controlada en un manejador. El traceback va a stderr

La validación de argumentos ocurre **antes** de ejecutar cualquier manejador, de modo que un argumento inválido nunca alcanza la lógica de negocio. Como jsonschema no está entre las dependencias permitidas por el enunciado, se implementó a mano el subconjunto de JSON Schema que los esquemas utilizan: type, required, enum, minLength, maxLength, pattern, minimum, maximum y additionalProperties.

El parámetro date de schedule_visit ilustra los dos lados de la frontera con el mismo argumento:

Entrada	Resultado	Razón
"25/08/2026"	-32602	Viola el pattern del esquema
"2026-02-30"	-32602	Tiene la forma correcta pero no existe en el calendario
"2020-01-01"	isError: true	Argumento válido, lo rechaza una regla de negocio

8.7 Ejemplos de intercambio

Todos capturados de un servidor en ejecución.

Error de dominio — la cuenta no existe. El intercambio fue exitoso:

```
{"jsonrpc": "2.0", "id": 5, "method": "tools/call", "params":
  {"name": "lookup_account", "arguments": {"account_id": "GT-99999"}}
```

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "\n\nerror\n": \"No se encontró ninguna cuenta con account_id='GT-99999'.\",
        \"account_id\": \"GT-99999\"
      }
    ],
    "isError": true
  }
}
```

Error de protocolo — falta un argumento obligatorio:

```
{
  "jsonrpc": "2.0",
  "id": 6,
  "method": "tools/call",
  "params": {
    "name": "check_service_status",
    "arguments": {}
  }
}

{
  "jsonrpc": "2.0",
  "id": 6,
  "error": {
    "code": -32602,
    "message": "account_id: is required",
    "data": {
      "field": "account_id"
    }
  }
}
```

8.8 Ejecución y conexión

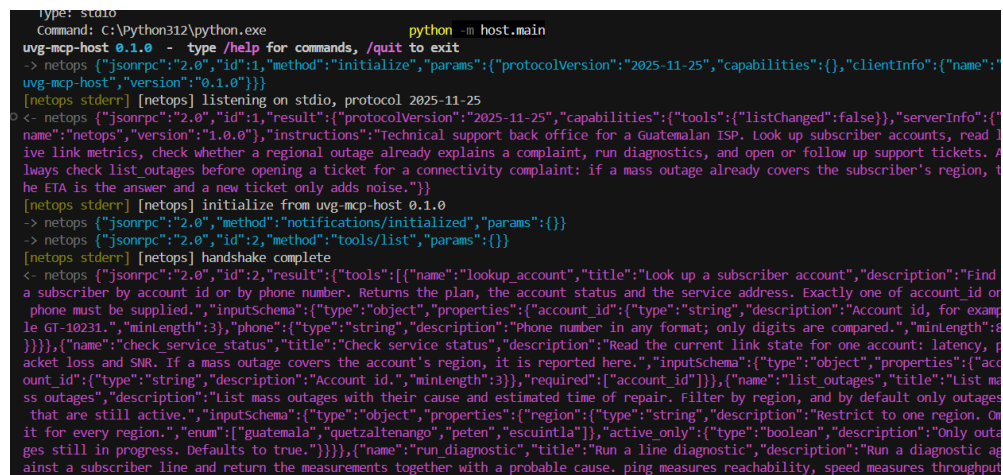
Desde la raíz del repositorio:

```
python -m servers.netops.stdio_server
```

Desde el host propio del proyecto, que ya lo declara en config/servers.json:

```
python -m host.main
/call netops__lookup_account {"account_id": "GT-10231"}
```

El host aplica espacio de nombres a las herramientas como <servidor>__<tool>. El separador es doble guion bajo y no punto ni dos puntos, porque los nombres de herramienta de la API de Anthropic deben cumplir `^[a-zA-Z0-9_-]{1,128}$`.



```
type: stdio
Command: C:\Python312\python.exe
uvg-mcp-host 0.1.0 - type /help for commands, /quit to exit
-> netops {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-11-25", "capabilities": {}, "clientInfo": {"name": "uvg-mcp-host", "version": "0.1.0"}}}
[netops stderr] [netops] listening on stdio, protocol 2025-11-25
<- netops {"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2025-11-25", "capabilities": {"tools": {"listChanged": false}}, "serverInfo": {"name": "netops", "version": "1.0.0"}, "instructions": "Technical support back office for a Guatemalan ISP. Look up subscriber accounts, read live link metrics, check whether a regional outage already explains a complaint, run diagnostics, and open or follow up support tickets. Always check list_outages before opening a ticket for a connectivity complaint: if a mass outage already covers the subscriber's region, the ETA is the answer and a new ticket only adds noise."}}
[netops stderr] [netops] initialize from uvg-mcp-host 0.1.0
-> netops {"jsonrpc": "2.0", "method": "notifications/initialized", "params": {}}
-> netops {"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
[netops stderr] [netops] handshake complete
<- netops {"jsonrpc": "2.0", "id": 2, "result": {"tools": [{"name": "lookup_account", "title": "Look up a subscriber account", "description": "Find a subscriber by account id or by phone number. Returns the plan, the account status and the service address. Exactly one of account_id or phone must be supplied.", "inputSchema": {"type": "object", "properties": {"account_id": {"type": "string", "description": "Account id, for example GT-10231.", "minLength": 3}, "phone": {"type": "string", "description": "Phone number in any format; only digits are compared.", "minLength": 8}}}, {"name": "check_service_status", "title": "Check service status", "description": "Read the current link state for one account: latency, packet loss and SNR. If a mass outage covers the account's region, it is reported here.", "inputSchema": {"type": "object", "properties": {"account_id": {"type": "string", "description": "Account id.", "minLength": 3}, "required": ["account_id"]}, {"name": "list_outages", "title": "List mass outages", "description": "List mass outages with their cause and estimated time of repair. Filter by region, and by default only outages that are still active.", "inputSchema": {"type": "object", "properties": {"region": {"type": "string", "description": "Restrict to one region. On it for every region.", "enum": ["guatemala", "quetzaltenango", "peten", "escuintla"]}, "active_only": {"type": "boolean", "description": "Only outages still in progress. Defaults to true."}}}, {"name": "run_diagnostic", "title": "Run a line diagnostic", "description": "Run a diagnostic against a subscriber line and return the measurements together with a probable cause. ping measures reachability, speed measures throughput"}]}}
```

Traza JSON-RPC del handshake en el host propio

Figura 1. El host imprime cada mensaje JSON-RPC enviado y recibido, que es el requisito de registro del enunciado. Se distingue el handshake completo: initialize, su resultado con el campo instructions, la notificación notifications/initialized que no lleva respuesta, y tools/list. Las líneas de stderr del servidor aparecen marcadas como tales y nunca se confunden con protocolo.

Desde Claude Code:

```
claude mcp add netops -- python -m servers.netops.stdio_server
claude mcp get netops
```

8.9 Persistencia

El almacenamiento separa lo inmutable de lo mutable, que es lo que mantiene los resultados reproducibles.

Entidad	Ubicación	Mutable
Cuentas	data/seed/accounts.json	No — versionado, solo lectura
Incidencias	data/seed/outages.json	No — versionado, solo lectura
Tickets y visitas	data/state.json	Sí — escritura atómica, fuera de git

La escritura del estado se hace mediante archivo temporal seguido de `os.replace`, que es atómico: una interrupción a medio camino no puede dejar el archivo truncado. Una prueba verifica que no queden archivos temporales huérfanos, y otra que la semilla nunca se escriba.

10. Conclusiones y comentarios sobre el proyecto

10.1 Estado actual

Métrica	Valor
Commits	12, uno por fase más los de documentación y correcciones
Líneas de código fuente	2,783 (host 1,756 · servidores 873 · utilidades 154)
Líneas de pruebas	1,822
Funciones de prueba	160, que ejecutan 198 casos por parametrización
Herramientas expuestas	7
Dependencias de terceros	5, ninguna relacionada con MCP

Las seis fases planificadas (F0 a F5) están completas y todos sus criterios de aceptación fueron verificados, incluido el de F4, que exigía que el modelo invocara las herramientas del servidor y respondiera con los resultados.

10.2 Decisiones de diseño que resultaron acertadas

Separar el transporte de la semántica. El transporte mueve mensajes completos y no sabe nada de MCP; `MCPClient` es dueño de la correlación y del handshake. Cuando el proyecto avance a HTTP, la clase nueva solo tiene que implementar cuatro métodos, y nada del cliente cambia. La misma idea aplica del lado del servidor: `core.py` no conoce JSON-RPC, y `stdio_server.py` es apenas un adaptador. Añadir `http_server.py` no moverá lógica de negocio.

Escribir jsonrpc.py como módulo puro, sin E/S. Eso permitió probarlo por completo antes de que existiera un solo subproceso, y es la razón por la que F1 precedió a F2. Cuando aparecieron problemas de concurrencia en F2, se sabía con certeza que los sobres no eran la causa.

Métricas determinísticas. Derivarlas de un hash en lugar de random significa que una demostración puede repetirse y dar lo mismo, y que las pruebas no necesitan tolerancias.

Un servidor de prueba que no comparte código con el cliente. Las pruebas de F2 levantan un servidor ficticio que usa json pelado en lugar de nuestro propio codificador. Si ambos extremos compartieran el codificador, un error en él se cancelaría solo y la prueba pasaría igual.

Aislar el proveedor de LLM detrás de una interfaz mínima. El agente solo necesita un objeto con un método create() que devuelva contenido y una razón de parada. Esa decisión no se tomó pensando en cambiar de proveedor, sino para poder probar el bucle con un modelo guionado. Cuando después hizo falta agregar un segundo proveedor, el costo fue **un archivo nuevo**: agent.py solo cambió para dejar de importar la traducción específica de Anthropic, y el registro, el transporte y el servidor no se tocaron. Una abstracción bien cortada se paga sola en el momento en que aparece el requisito que no se había previsto.

10.3 Dificultades encontradas

La concurrencia del transporte fue lo más costoso, tal como anticipaba el enunciado. El problema no fue escribir el hilo lector sino los caminos de falla: si el servidor muere, cada petición pendiente debe fallar con una excepción en lugar de quedar colgada para siempre. Eso se resolvió fallando explícitamente todos los Future pendientes al detectar EOF, y hay pruebas que matan al servidor a propósito y exigen que el llamador reciba un error.

La codificación en Windows exigió forzar UTF-8 tanto en el proceso hijo como en los flujos del servidor. El valor por defecto cp1252 corrompe cualquier carga útil con acentos, y los datos semilla contienen varios.

Dos fallas de conformidad aparecieron solo al manejar el servidor desde afuera.

Conducirlo desde PowerShell, en lugar de desde nuestro propio host, reveló lo que las 151 pruebas existentes hasta ese momento no detectaban, porque host y servidor compartían supuestos:

1. notifications/initialized marcaba la sesión como inicializada aunque initialize nunca hubiera corrido, de modo que un cliente podía saltarse el handshake y aún así alcanzar tools/list. La notificación *confirma* un handshake; no puede iniciarlo.
2. Una marca de orden de bytes al inicio de la línea hacía fallar el parseo con -32700. JSON es UTF-8 y la marca sobra, pero varias herramientas la emiten y el RFC 8259 permite ignorarla.

Ambas tienen prueba de regresión. La lección es que las pruebas escritas contra la propia implementación tienen un punto ciego estructural: solo un cliente ajeno lo revela.

Conectar Claude Desktop en Windows tuvo dos obstáculos no documentados. El primero es que la instalación desde Microsoft Store empaqueta la aplicación como MSIX, lo cual virtualiza %APPDATA%: el archivo de configuración que la aplicación lee realmente

no está en %APPDATA%\Claude\ sino en %LOCALAPPDATA%\Packages\Claude_pzs8srxjxfjcc\LocalCache\Roaming\Claude\, de modo que escribir en la ruta documentada no surte ningún efecto. El segundo es que cerrar la ventana no recarga la configuración, porque la aplicación queda residente; un segundo lanzamiento detecta la instancia viva y termina de inmediato registrando `Not main instance, returning early from app ready`. Hay que salir desde el ícono de la bandeja o terminar todos los procesos. Ambos detalles quedaron documentados en SPEC.md sección 7.5.

Una refactorización dejó una llamada muerta que ninguna prueba detectó. Al generalizar la configuración para admitir varios proveedores se renombró un atributo, y quedó una lectura del nombre viejo en el manejador de preguntas del CLI. Escribir cualquier cosa que no empezara con / terminaba la sesión con un `AttributeError`. Las ciento ochenta y tres pruebas de ese momento pasaban en verde, porque ninguna manejaba el CLI con el asistente sin configurar: todas o construían un agente, o llamaban las piezas por separado. El camino más obvio del programa —abrirlo y escribir una pregunta— no estaba cubierto. Se agregó `tests/test_cli_config.py` para cerrar ese hueco. La lección es que la cobertura alta no implica cobertura de los caminos que un usuario recorre primero.

10.4 Lo que falta

Del avance no queda nada pendiente: los seis criterios de aceptación fueron verificados, el último de ellos contra un modelo real, como se documenta en 10.6.

Sí conviene declarar con precisión un punto que no se ejercitó. El adaptador para la **API de Anthropic** está implementado y cubierto por pruebas, pero ninguna llamada real lo atravesó: la cuenta autentica correctamente y no tiene saldo, de modo que el único intento devolvió `400: credit balance is too low`. El bucle se verificó entonces contra Groq, que hospeda modelos de pesos abiertos y ofrece un nivel gratuito. Como ambos adaptadores implementan la misma interfaz y el bucle no distingue cuál responde, lo que quedó sin ejercitar es la traducción específica de un proveedor, no el mecanismo.

Queda también pendiente todo lo correspondiente al proyecto final: el transporte HTTP remoto, el despliegue con Docker y Cloud Run, la captura con Wireshark y la integración de servidores oficiales de terceros.

10.5 Validación con hosts de terceros

El enunciado sugiere conectar el servidor a **Claude Desktop** como prueba de conformidad. Se realizó, contra la versión 1.32885.1.0, y el resultado quedó registrado en los logs de ambos extremos.

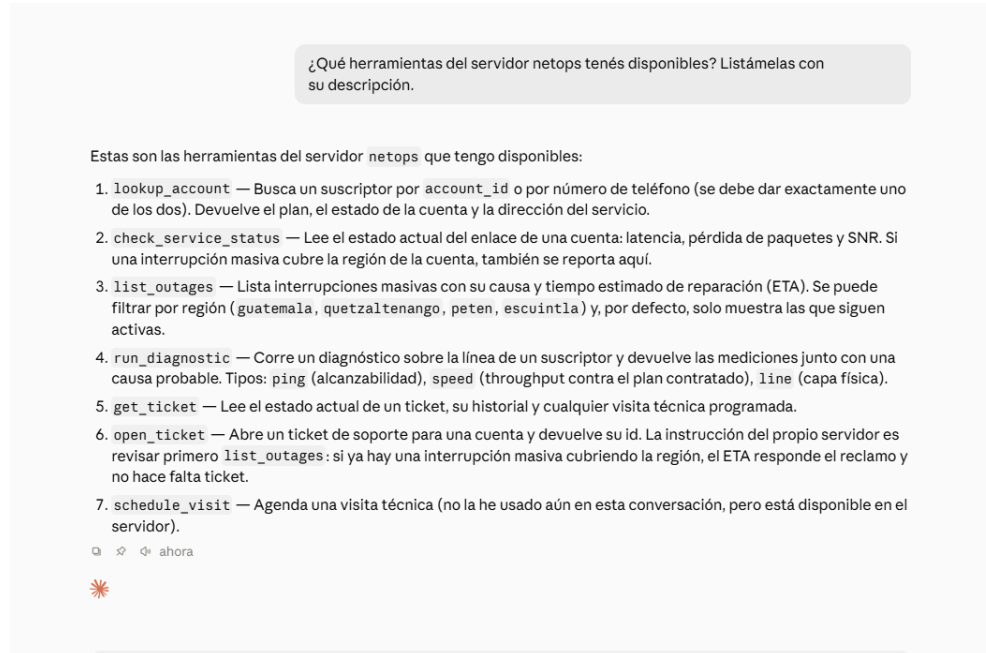
Del lado del host:

```
[LocalMcpServerManager] netops negotiated protocol version: 2025-11-25
[LocalMcpServerManager] Connected to netops (7 tools)
[localMcpBridge] announcing netops: 7 tool(s)
```

Del lado del servidor, en el stderr que el propio host capturó:

```
[netops] listening on stdio, protocol 2025-11-25
[netops] initialize from claude-ai 0.1.0
[netops] handshake complete
```

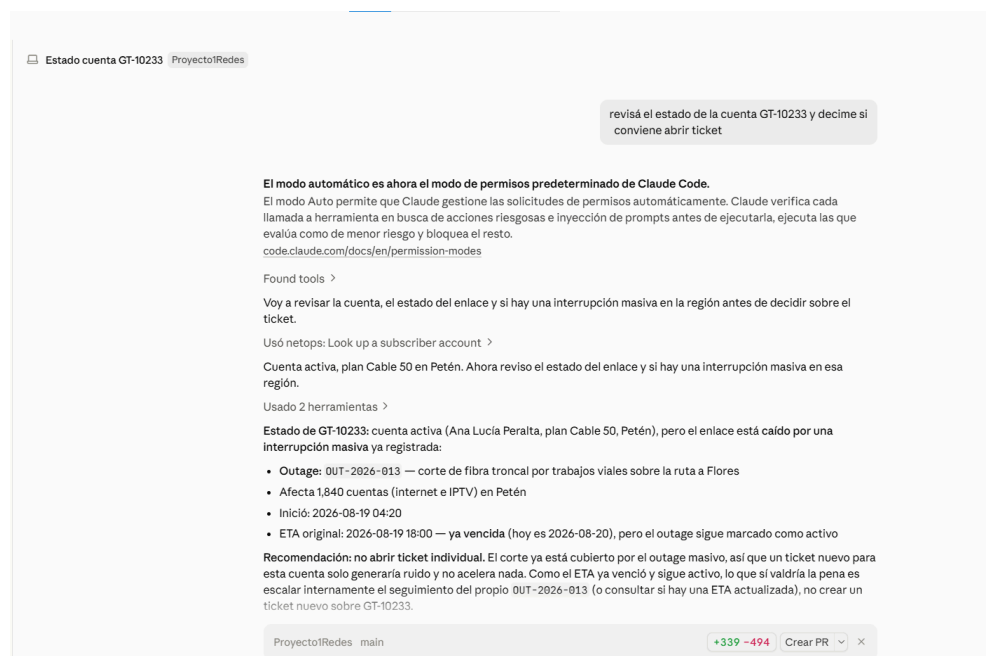
La línea `initialize from claude-ai 0.1.0` es significativa: no es un mensaje genérico sino el servidor leyendo el campo `clientInfo` que Claude Desktop envió en el handshake. Es la implementación propia procesando datos reales de un cliente ajeno.



El modelo enumera las siete herramientas descubiertas del servidor netops

Figura 2. Al preguntársele qué herramientas tiene disponibles, el modelo enumera las siete con sus descripciones, que son exactamente las que el servidor publica en `tools/list`. Nótese la entrada de `open_ticket`: el modelo cita la guía del propio servidor —revisar `list_outages` primero— sin que nadie se la haya mencionado en la conversación. Esa guía llegó por el campo `instructions` del handshake.

Más allá del handshake, se verificó el uso efectivo de las herramientas desde ese host. Ante la consulta “*revisá el estado de la cuenta GT-10233 y decime si conviene abrir ticket*”, el modelo encadenó tres herramientas por su cuenta y concluyó que no correspondía abrir un ticket individual porque una incidencia masiva ya cubría a la abonada.



El modelo encadena tres herramientas de netops y emite una recomendación

Figura 3. El modelo decide por su cuenta qué herramientas invocar y en qué orden, y razona sobre los resultados. También advierte que el ETA de la incidencia ya venció mientras su estado sigue activo, una inconsistencia de los datos que ninguna instrucción le pidió buscar.

Validaciones adicionales. El servidor también conecta desde **Claude Code** (claude mcp get netops reporta Status: ✓ Connected), y se escribió tools/conformance_check.py, un cliente independiente que usa solo la biblioteca estándar y no importa nada de host/, que verifica 19 aserciones sobre el ciclo de vida, las herramientas, los errores y el apagado. Tres hosts distintos, dos de ellos de producción y ninguno con conocimiento del código de este repositorio.

10.6 El bucle agéntico de extremo a extremo

Lo anterior valida el **servidor**. Esta sección valida el **host propio**, que es la otra mitad del entregable: el enunciado pide un servidor MCP *“que sea utilizado por su chatbot”*, y ese chatbot es host/main.py.

Ante la misma consulta, con el modelo openai/gpt-oss-120b servido por Groq, el bucle de host/agent.py produjo esta traza:

Model openai/gpt-oss-120b via groq ready.

```
> El cliente GT-10233 reporta que no tiene internet desde la madrugada...
  calling netops__lookup_account {"account_id": "GT-10233"}
  rate limited, waiting 16s
  calling netops__list_outages {"region": "peten"}
  rate limited, waiting 39s
```

Vale la pena detenerse en el segundo argumento. El modelo no recibió la región en ningún momento: la leyó del resultado de lookup_account y la usó para filtrar list_outages. Eso es encadenamiento real, no dos llamadas independientes. Sobre esos resultados construyó la recomendación de no abrir ticket, que es exactamente la guía que el servidor publica en instructions.

Los avisos de espera son el cliente respetando los límites del nivel gratuito. El proveedor responde 429 indicando cuántos segundos faltan, y el adaptador espera ese tiempo y reintenta en lugar de abandonar el turno. Los siete esquemas de herramientas ocupan suficientes tokens como para que un turno de varios pasos roce el límite con frecuencia.

```
ption": "Arrival window.", "enum": [{"start": "08:00-12:00", "end": "12:00-16:00", "required": ["ticket_id", "date", "time_window"]}]]]]
Connected to 1/1 server(s), 7 tool(s) available.
Model openai/gpt-oss-120b via groq ready. Ask a question, or use /call to invoke a tool directly.
> El cliente GT-10233 reporta que no tiene internet desde la madrugada. Revisá qué está pasando y decime si conviene abrir ticket.
  calling netops__check_service_status {"account_id": "GT-10233"}
  <- netops {"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "check_service_status", "arguments": {"account_id": "GT-10233"}}}
  <- netops {"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text", "text": "\n \"account_id\": \"GT-10233\", \n \"account status\": \"\n \"active\", \n \"region\": \"peten\", \n \"link state\": \"down\", \n \"reason\": \"mass outage\", \n \"detail\": \"Corte de fibra troncal\n por trabajos viales sobre la ruta a Flores\", \n \"outage_id\": \"OUT-2026-013\", \n \"eta\": \"2026-08-19T18:00:00-06:00\", \n \"metric\n s\": { \n \"latency_ms\": 31, \n \"packet_loss_pct\": 0.6, \n \"snr_db\": 34.2, \n \"downstream_mbps\": 42.0, \n \"upstream_mbp\n s\": 23.0 \n } \n } ]}], \"isError\": false}}
  rate limited, waiting 18s
El enlace de GT-10233 está caído por un corte de fibra troncal en la ruta a Flores (outage OUT-2026-013). Es una interrupción masiva y se
espera que se restablezca a las 18:00 del 19 de agosto de 2026. No es necesario abrir un ticket; basta con esperar la reparación.
>
>
```

El bucle agéntico propio con la traza de protocolo visible

Figura 4. Otra ejecución del mismo bucle, esta vez con la traza de protocolo activada, de modo que la imagen muestra las dos capas a la vez. Arriba, el modelo decide llamar check_service_status; en medio, el tools/call que el host emite y la respuesta del

servidor con `isError: false`; abajo, la respuesta en lenguaje natural construida sobre ese resultado. Entre ambos, el aviso `rate limited, waiting 18s`: el cliente absorbe el límite del nivel gratuito y continúa el turno en lugar de abandonarlo.

Esto cierra el criterio de aceptación de F4 —“*El LLM invoca tools del servidor y responde con los resultados*”— con las tres piezas propias en la cadena: el bucle, el cliente MCP y el servidor.

10.7 Comentario sobre MCP

Lo más valioso de implementar el protocolo a mano fue entender que MCP resuelve un problema de acoplamiento más que uno técnico. El protocolo en sí es JSON-RPC 2.0 con un handshake y tres métodos; nada de eso es difícil. Lo que aporta es que el servidor publica sus propias capacidades —nombres, esquemas y hasta instrucciones dirigidas al modelo— de modo que el host no necesita saber nada sobre el dominio del servidor, y el servidor no necesita saber qué modelo lo va a usar. Escribir el cliente y el servidor en el mismo repositorio y luego ver a Claude Desktop conectarse a ese servidor sin modificar una línea deja esa idea clara de una forma que leer la especificación no logra.

El desacoplamiento resultó ir en las dos direcciones, y eso no estaba previsto. El servidor terminó siendo consumido por tres hosts que no lo conocían, y el host propio terminó ejecutando dos proveedores de modelo distintos sin que el bucle notara la diferencia. Ninguna de las dos cosas exigió rediseñar nada, porque en ambos casos el límite estaba puesto donde correspondía: el servidor no sabe quién lo llama, y el bucle no sabe qué modelo responde. Es la misma lección vista desde los dos extremos.

La distinción entre error de protocolo y error de herramienta fue el punto donde más se aprendió. Es tentador tratar “la cuenta no existe” como un error, y el protocolo insiste en que no lo es: es un resultado exitoso con contenido que el modelo debe leer. Esa separación es lo que permite que un modelo se recupere solo de un argumento equivocado, y es la razón por la que el bucle agéntico devuelve cada falla de herramienta al modelo en lugar de propagarla como excepción.